

Experiment 'p_6_1_c_process_array_reverse_order_array'

Results

June 4, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing assume : 0
Responses containing expect : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_6_1_c_process_array_reverse_order_array

Natural language statement: Write a method that takes an array of ten integers and then returns all elements in reverse order.

Method signature: p_6_1_c_process_array_reverse_order_array(inputs: array<int>) returns (reversed: array<int>)

Ensures

- `reversed.Length == inputs.Length`
- `forall i :: 0 <= i < reversed.Length ==> reversed[i] == inputs[inputs.Length - 1 - i]`

Requires

- `inputs.Length == 10`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes an array of ten integers and then returns all elements in
↪ reverse order.

The signature should be:

p_6_1_c_process_array_reverse_order_array(inputs: array<int>) returns (reversed: array<
↪ int>)

The method should respect the following contract:

requires inputs.Length == 10, ensures reversed.Length == inputs.Length, ensures forall i
↪ :: 0 <= i < reversed.Length ==> reversed[i] == inputs[inputs.Length - 1 - i]

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn09UE89fJkauMiJEnWHZlmIuZ6PB

Model: gpt-5.5-2026-04-23

Created at: 1780571028

Finish reason: stop

Usage: 344 tokens in, and 355 tokens out

```
{
  reversed := new int[inputs.Length];
  var i := 0;
  while i < reversed.Length
    invariant reversed.Length == inputs.Length
    invariant 0 <= i <= reversed.Length
    invariant forall j :: 0 <= j < i ==> reversed[j] == inputs[inputs.Length - 1 - j]
  {
    reversed[i] := inputs[inputs.Length - 1 - i];
    i := i + 1;
  }
}
```

Final Program

```
method p_6_1_c_process_array_reverse_order_array(inputs: array<int>) returns (reversed:
  ↪ array<int>)
```

```

    requires inputs.Length == 10
    ensures reversed.Length == inputs.Length
    ensures forall i :: 0 <= i < reversed.Length ==> reversed[i] == inputs[inputs.
        ↪ Length - 1 - i]
{
    reversed := new int[inputs.Length];
    var i := 0;
    while i < reversed.Length
        invariant reversed.Length == inputs.Length
        invariant 0 <= i <= reversed.Length
        invariant forall j :: 0 <= j < i ==> reversed[j] == inputs[inputs.Length - 1 - j]
    {
        reversed[i] := inputs[inputs.Length - 1 - i];
        i := i + 1;
    }
}

```

Total Token Usage

Input tokens: 344

Output tokens: 355

Reasoning tokens: 231

Sum of 'total tokens': 699

Experiment Timings

Overall Experiment started at 1780571020464, ended at 1780571062243, lasting 41779ms (41.78 seconds)

Iteration #1 started at 1780571020481, ended at 1780571062240, lasting 41759ms (41.76 seconds)